DATA PIPELINE CASE STUDY

ON-PREMISE $0.00 OPEX

MovieLens 32M Relational Joining Pipeline

Low-Resource High-Throughput R Tidyverse Engineering Architecture

High-efficiency relational data pipeline processing **32,000,000+** movie ratings and tagging records using localized memory optimizations under strict physical hardware boundaries.

DATASET VOLUME	PROCESSING ENGINE	CLOUD EXPENDITURE
32,000,000+	R Tidyverse	$0.00 OPEX

SYSTEM ENVIRONMENT

Hardware Specifications & Execution Limits

Core i3 @ 1.2GHz

Budget dual-core processor requiring vectorized stream operations to prevent thread throttling.

4.00 GB RAM Limit

Strict physical RAM ceiling requiring integer coercion to aggregate `32M` records safely.

~2.0 GB OS Overhead

Windows system background services leave `~1.9 GB` net executable heap space.

$0.00 Cloud OPEX

Fully processed locally on-premise without external cloud infrastructure costs.

Memory Guard Protocol: Pre-filtering corrupted rows prevents heap overflows.

✓ OPERATIONAL 100%

ARCHITECTURE FLOW

Data Pipeline Stage Flowchart & Execution Details

1. Ingestion

Streams raw rating & tag CSV files into memory buffers via C++ fast parsers.

2. Cleaning

Strips duplicates and enforces strict 32-bit integer conversion on ID fields.

3. Joining

Executes indexed inner join matching rating vectors with grouped movie tags.

4. Persistence

Projects target metadata fields and writes clean CSV payload directly to disk.

Peak executable heap RAM remains strictly under `1.85 GB` throughout execution.

</> PIPELINE SCRIPT

Full Pipeline Script Implementation Code

New_dataset_movielens_32M.R

```r
library(tidyverse); library(readr)

# 1. Ingest & Sanitize Ratings Data
cleaned_rating_data <- read_csv(file.choose()) %>% distinct() %>%
  mutate(userId = as.integer(userId), movieId = as.integer(movieId), rating = as.integer(rating)) %>%
  filter(userId > 0 & movieId > 0 & rating > 0)

# 2. Ingest & Group Tags Metadata
cleaned_tags_data <- read_csv(file.choose()) %>% distinct() %>%
  mutate(userId = as.integer(userId), movieId = as.integer(movieId)) %>%
  filter(userId > 0 & movieId > 0) %>% group_by(movieId) %>%
  summarise(Total_Tags = n(), .groups = 'drop')

# 3. Inner Join & Export Cleaned Payload
final_report <- cleaned_rating_data %>% inner_join(cleaned_tags_data, by = 'movieId') %>%
  distinct() %>% select(userId, movieId, rating, Total_Tags)

write.csv(final_report, 'New_dataset_movielens_32M_data_joining_pieline_data.csv', row.names = FALSE)
```

⚡ STAGES 1 & 2

Data Ingestion & Type Sanitization Logic

```
# Ingestion & Sanitization via readr & dplyr
cleaned_rating_data <- read_csv(file.choose()) %>% distinct() %>%
  mutate(userId = as.integer(userId), movieId = as.integer(movieId), rating = as.integer(rating)) %>%
  filter(userId > 0 & movieId > 0 & rating > 0)
```

C++ Streaming

readr::read_csv streams raw CSV rows directly into memory via C++ parsers.

Integer Coercion

Casts double precision fields to 32-bit integers, cutting memory usage by 50%.

Boundary Filtering

filter() purges corrupted IDs and invalid zero/negative ratings upfront.

STAGE 3

Relational Inner Join & Tag Summarization Engine Flow

```r
# Summarize tags per movie ID & execute relational join
cleaned_tags_data <- tags_raw_data %>% distinct() %>%
  filter(userId > 0 & movieId > 0) %>%
  group_by(movieId) %>% summarise(Total_Tags = n(), .groups = 'drop')

join_report <- cleaned_rating_data %>% inner_join(cleaned_tags_data, by = 'movieId')
```

Grouped Tag Rollup

Aggregates raw tag occurrences into a compact `Total_Tags` metric per movie.

Inner Join Execution

Merges ratings and tag summaries matching strictly on indexed movieId primary keys.

STAGE 4

Feature Selection & Disk Persistence Output Export

```r
# Project columns & export final clean CSV payload
final_report <- join_report %>% distinct() %>%
  select(userId, movieId, rating, Total_Tags)

write.csv(final_report, 'New_dataset_movielens_32M_data_joining_pieline_data.csv', row.names = FALSE)
```

Selective Projection

select() retains only required analytical columns to minimize export file size.

Clean CSV Export

Persists the clean joined data payload directly to disk without redundant row indices.

OPEN SOURCE & CONNECT

Explore the Data Engineering Repository

Inspect complete R pipeline routines, transformation benchmarks, and interactive BI dashboards on GitHub or LinkedIn.



GITHUB REPOSITORY

github.com/talkmridul-bit



LINKEDIN PROFILE

linkedin.com/in/niaz-shoyab-495061b8